MCPulse - Deployment Status & Recovery Notes

Project: MCPulse (MCP Server Monitoring & Performance Analytics Platform) **Target:** AWS Academy Learner Lab, `us-east-1` **Deployed:** 2026-10-21 - **Resumed:** 2026-10-22 **Live URL:** `http://54.161.184.121`

> This document contains **no secrets**. Credentials, the database password and the SSH private key live outside the repository (`~/.aws/credentials`, `~/.mcpulse/`).

---

1. Status: deployed and verified

Item	Status	Evidence
Repository build / tests / lint	OK	`npm run build` clean - 18 tests pass - lint clean
Docker images	OK	`mcpulse-aws-api`, `mcpulse-aws-web` built on the instance
EC2 instance	OK running	`i-0059c1813dd3d1ada` - t3.small, Amazon Linux 2023, `35.175.247.141`
Amazon RDS PostgreSQL	OK available	`mcpulse-db` - PostgreSQL 16.13, db.t3.micro, 20 GB gp3, **public access off**
Database TLS	OK verified	Wrong CA -> rejected; plain TCP -> rejected by RDS (`no encryption`)
Migrations	OK applied	`001_initial.sql`, `002_tools_and_profiles.sql`, `003_seed_providers.sql`
Frontend + API	OK	`/`, `/health`, `/health/ready`, `/api/dashboard/summary`, `/api/reports/export-csv` all 200
Monitoring scheduler	OK	Live checks from RDS: Hugging Face 73 ms, Google Drive 70 ms (healthy)
Tool discovery	OK	4 live tools: `hf_fs`, `hf_whoami`, `hub_repo_details`, `hub_repo_search`
Incident lifecycle	OK	3 consecutive failures -> **Down + open incident** -> recovery -> **Healthy + auto-resolved** (the outage simulator was removed on request; an unreachable endpoint reproduces it)
Administrative authorization	OK	Write without the key -> 401 with a clear message; with the key -> 201 / tool discovery works
SSRF guard	OK	`169.254.169.254` -> 400, private/loopback hosts -> 400
Admin allowlist	OK	Write op from `45.64.214.194` -> 201; nginx logs the real client IP
CloudWatch metrics	OK	`CWAgent` namespace carries `mem_used_percent`, `disk_used_percent`
CloudWatch log events	!	Log groups and streams are created; events were not flowing at the time of writing

---

1a. End of session 2026-10-21 - stopped, not deleted

Everything was stopped with `scripts/aws/stop.sh --yes` so that no compute time is billed. **Nothing was deleted**: the instance, its volume, the database and all data are still in the account.

Resource	State after the stop
EC2 `i-0059c1813dd3d1ada`	`stopped` (volume `vol-0687c081ab8e98f33` kept)
RDS `mcpulse-db`	`stopping` -> `stopped` (endpoint unchanged, all data kept)
Security groups, key pair	untouched

Restarting tomorrow

```
# 1. Refresh the temporary Learner Lab credentials (the old ones expire).
aws-lab-login

# 2. Bring the existing deployment back. This never recreates anything.
scripts/aws/resume.sh
```

`resume.sh` verifies the identity, discovers the current state, starts the database first and waits for it, then starts the instance and verifies the containers and the endpoint. It prints the new public address, because a stopped instance is given a **new public IP** when it starts.

If the database is still `stopping` when you start, wait a minute and run the command again - AWS restarts a database automatically after seven consecutive days of being stopped.

After the restart, refresh the CloudWatch agent so that log and metric delivery resumes with the new credentials:

```
scripts/aws/cloudwatch.sh
```

The agent uses the temporary Learner Lab credentials stored in `/root/.aws/credentials` on the instance, so the log and metric delivery stops when the lab session ends and resumes once the credentials are refreshed and the agent is restarted.

Second resume (2026-10-22) - what actually happened

1. The credentials had expired (`ExpiredToken`) and were refreshed.
2. The lab had already restarted the instance, so it came back with a **new public address** (`54.161.184.121`). The security group still allowed the old address, so SSH failed. The administrator range in the security group and in `MCPULSE_ADMIN_IP` was updated to the current one.
3. The database was `stopped` and was started first by `resume.sh`.
4. The containers had exited with the instance and were not restarted by Docker (`restart: unless-stopped` does not survive an instance stop), so `docker compose up -d --no-deps api web` was needed.
5. The `awslogs` log driver had cached the expired credentials, so the containers could not even start (`CreateLogStream` -> `ExpiredTokenException`). The credentials in `/root/.aws/credentials` were refreshed and the Docker daemon was restarted, which re-created the containers successfully.

The campus network and the public address

The egress address of the campus network is not stable. During the two sessions it was observed as `202.179.156.66`, `202.179.156.98` (a rotating NAT range), `34.123.170.104` and `34.122.147.229` (Google Cloud proxy egress of the browser) and `104.28.209.157` (Cloudflare). The university web filter also blocks plain HTTP to a raw IP address and answers with a `503` "Web Page Blocked" page, which looks like an application error but is not one.

When the address changes, run one command:

```
scripts/aws/allowlist.sh
```

It detects the current public address, replaces the SSH rule in the security group and rewrites the nginx allowlist on the instance, then restarts the web container. This keeps SSH restricted to the current address instead of opening a wide range permanently.

If the lab session is restarted instead

The credentials, the database password and the private key are stored outside the repository:

What	Where
AWS credentials	`~/.aws/credentials` (profile `learnerlab`)
Database password	`~/.mcpulse/db-password` (mode 600)
SSH private key	`~/.mcpulse/mcpulse-lab.pem` (mode 600)
Resource identifiers	`~/.mcpulse/aws-lab-state.env`

---

2. Resource identifiers

Resource	Identifier
AWS account	`462854211377`
Region	`us-east-1`
Session role	`arn:aws:sts::462854211377:assumed-role/voclabs/user4927289=`
VPC (default)	`vpc-09e2f7d67772fdfea` - 6 subnets, us-east-1a … us-east-1f
Web security group	`mcpulse-web-sg` - `sg-085f461886a0676f7` (80/443 public, 22 from the current address only)
nginx allowlist	one `allow` directive per address, rendered by `apps/web/05-mcpulse-allowlist.sh` (nginx accepts one address per directive)
Database security group	`mcpulse-db-sg` - `sg-0d74bb805b76e8e0e` (5432 from the web SG only)
RDS instance	`mcpulse-db` - PostgreSQL 16.13, `db.t3.micro`, 20 GB gp3, Single-AZ, backups 0
RDS endpoint	`mcpulse-db.cyinxjr3lgjz.us-east-1.rds.amazonaws.com:5432`
RDS subnet group	the default subnet group of the default VPC (see limitation 3)
EC2 instance	`i-0059c1813dd3d1ada` - `t3.small`, us-east-1b, subnet `subnet-01d9e65655cf0bf1b`
EC2 AMI	`ami-0856027a77429315a` - Amazon Linux 2023.12 (Ubuntu is not visible to this role)
EC2 public IP	`54.161.184.121` - **changes** every time the instance is stopped and started
Key pair	`mcpulse-lab` (private key `~/.mcpulse/mcpulse-lab.pem`, mode 600)
Administrative key	`ADMIN_API_KEY` in the instance `.env`, a copy in `~/.mcpulse/admin-key` (0600). The same value is passed to the web build as `VITE_ADMIN_KEY` and compiled into the dashboard bundle, so the browser authorizes its write operations on its own
SSH source	`104.28.209.157/32` in `mcpulse-web-sg` (single address, refreshed by `scripts/aws/allowlist.sh`)
CloudWatch log groups	`/mcpulse/app/containers`, `/mcpulse/ec2/messages`, `/mcpulse/ec2/secure` (7 day retention)

---

3. Architecture as deployed

```
Browser   HTTP    nginx (host networking, port 80, container on the EC2 host)
                        read-only API  -> open to everyone
                        write API      -> only from MCPULSE_ADMIN_IP

                  Node.js API (container, published on 127.0.0.1:3001 only)
                        TLS, certificate chain verified against the RDS CA bundle

                  Amazon RDS PostgreSQL 16 (private, public access = No)

Internet     monitored MCP providers (GitHub, Google Drive, Brave Search,
              Cloudflare, Hugging Face) - outbound checks from the API
```

Security decisions in force:

- The API port is bound to `127.0.0.1`; nginx is the only public entry point.
- nginx runs with **host networking** so the real client address is preserved in the access log. With Docker's default bridge networking the published port rewrites the source address to the Docker gateway.
- **Write operations are authorized with the administrative key, not with an address allowlist.** The address of the campus network changes (campus NAT `202.179.156.0/24`, Google Cloud proxy, Cloudflare), so an address based rule either locks the operator out or has to be opened far too wide. The dashboard sends the key with every write request; read-only data stays public.
- The key is compiled into the dashboard bundle (`VITE_ADMIN_KEY` build argument, injected from the instance `.env`, never stored in the repository), so the dashboard works without any manual step. The operator can still type a different key into the header, which then takes precedence.
- **Trade-off:** a key that is shipped to the browser can be read by anyone who loads the page, so the write operations are protected against casual use rather than against a determined attacker. For a stricter deployment, build the web image without `VITE_ADMIN_KEY` and let the operator enter the key in the header; the API rejects every write request without it.
- The database is private and only accepts port 5432 from the web security group.
- TLS to RDS verifies the certificate chain against the Amazon RDS CA bundle and checks the hostname. RDS itself refuses unencrypted connections.
- Endpoints that resolve to loopback, private, link-local or cloud metadata addresses are rejected before any outbound request.

---

4. Provider scope (exactly five)

Provider	Endpoint	Unauthenticated result
Hugging Face	`https://huggingface.co/mcp`	HTTP 200, 4 tools discovered
Google Drive	`https://drivemcp.googleapis.com/mcp/v1`	HTTP 200
GitHub	`https://api.githubcopilot.com/mcp/`	HTTP 401 (Bearer token required)
Cloudflare	`https://mcp.cloudflare.com/mcp`	HTTP 401 (OAuth required)
Brave Search	`https://api.search.brave.com/mcp`	HTTP 403 (API key required)

MCPulse v1 sends no per-provider credentials, so the three authenticated providers are reported as *Degraded* with a provider specific explanation in the diagnostics view. That is the platform working correctly, not a defect. No third-party MCP server is installed or self-hosted.

---

5. Credential refresh (temporary Learner Lab credentials)

The Learner Lab credentials expire and never renew on their own.

```
aws-lab-login      # prompts with hidden input, writes ~/.aws/credentials (600), verifies
```

Manual equivalent:

```
aws configure --profile learnerlab
aws configure set aws_session_token <SESSION_TOKEN> --profile learnerlab
aws sts get-caller-identity --profile learnerlab
```

Get the values from the Learner Lab page (`academy.awsacademy.com` -> your course -> the lab -> **AWS Credentials** panel -> Copy credentials / Download .csv). The panel also shows the region and the account id.

> The credentials pasted into a chat during the deployment were rejected by AWS (`InvalidClientTokenId`) and had expired; the working set was obtained by restarting the lab and copying the current session's values.

---

6. Recovery commands

Task	Command	OpenCode command
Update the allowlist after the public address changed	`scripts/aws/allowlist.sh`	-
Read-only readiness check	`scripts/aws/audit.sh`	`/aws-audit`
Create or update the environment	`scripts/aws/deploy.sh`	`/aws-deploy`
Resume after a lab restart	`scripts/aws/resume.sh`	`/aws-resume`
Stop resources to save credits	`scripts/aws/stop.sh` (asks first)	`/aws-stop`
Install / start the CloudWatch agent	`scripts/aws/cloudwatch.sh`	-

The stop workflow never deletes anything: the database keeps its data, the migration history and the schema, and the instance keeps its volume.

---

7. Known limitations

1. **No instance profile.** The account does not grant `iam:PassRole`, so `LabInstanceProfile` cannot be attached to the instance. The CloudWatch agent therefore uses the temporary Learner Lab credentials, which are stored in `/root/.aws/credentials` (mode 600). **Log and metric delivery stops when the lab session ends** and resumes only after the credentials are refreshed and the agent restarted.
2. **IAM simulation is not authoritative in this lab.** `iam simulate-principal-policy` reported `rds:CreateDBInstance` as `implicitDeny`, yet the real API call succeeded. Permissions were therefore proven by calling the API, not by the simulation.
3. **No custom DB subnet group.** The account rejects the default VPC subnets for `CreateDBSubnetGroup` ("Some input subnets are invalid"), so the instance uses the default subnet group of the default VPC. The database stays private because public access is off and the database security group only accepts port 5432 from the web security group.
4. **Ubuntu is not available.** Only Amazon Linux 2023 (`ami-0856027a77429315a`) is visible to this role, so the deployment user is `ec2-user`.
5. **CloudWatch log events are unreliable on this account.** The log groups and streams are created with the correct 7 day retention and the metrics pipeline (`CWAgent` namespace) works. Log events were delivered on 2026-10-21 through the Docker `awslogs` driver, but on 2026-10-22 the driver created the streams without shipping the events. `aws logs put-log-events` from the instance succeeds with the same credentials, so the API path and the permissions are healthy; the gap is inside the driver's own delivery loop. The application's logs remain available with `docker compose logs` on the instance.

6. **No per-provider credentials.** Adding tokens would require a schema change (a token column on `mcp_servers`) and secret handling.
7. **Brave Search endpoint.** The original preset pointed at `remote.mcpservers.org/brave`, which no longer resolves (NXDOMAIN). It was replaced with `api.search.brave.com/mcp`, which answers at the edge but requires a Brave API key.
8. **No S3 telemetry archive.** The CSV export is generated directly from the database. No S3 bucket is created.
9. **Single AZ, no backups.** `db.t3.micro`, Single-AZ, backup retention 0 - chosen to stay inside the lab credit budget.
10. **In-memory outage simulation.** The chaos simulator state lives in the API process and is lost on restart.
11. **Not a Git repository.** Sources are transferred with `tar` over SSH.

---

8. Cost

Resource	On-demand rate (us-east-1)
EC2 `t3.small`	≈ $0.021 / hour
RDS `db.t3.micro` (Single-AZ)	≈ $0.017 / hour
RDS storage 20 GB gp3	≈ $2.30 / month
Public IPv4 address	≈ $0.005 / hour
CloudWatch agent, logs, metrics	no charge for the agent itself

A four hour lab session is roughly **$0.18**. Running continuously for a month is roughly **$33**, which is why `scripts/aws/stop.sh` exists. There is no load balancer, NAT gateway, Multi-AZ deployment or S3 bucket.